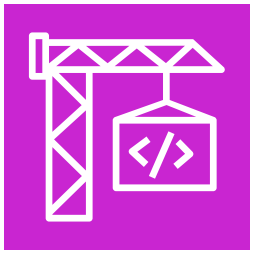
AWS CodeBuild and CodeDeploy: CI/CD Fundamentals

Understand how AWS CodeBuild automates building and testing your code using buildspec files, and how AWS CodeDeploy streamlines application deployment to EC2, Lambda, and ECS. Explore key components, workflows, and best practices for managing CI/CD processes effectively within AWS.

AWS provides a suite of tools designed to support CI/CD processes for cloud applications. Among these, AWS CodeBuild and AWS CodeDeploy are in-scope for the AWS Certified Developer – Associate (DVA-C02) exam. These services help automate the build and deployment stages of the application life cycle, enabling developers to deliver software reliably and efficiently.

AWS CodeBuild: Automating code compilation and testing

AWS CodeBuild is a fully managed build service that compiles source code, runs unit tests, and produces deployable artifacts. It eliminates the need for developers to manage their own build infrastructure.



Ask

To understand the working of code build, let's first discuss its core components:

Buildspec

The buildspec is a YAML file named `buildspec.yml` that is included in the root of the source code. It contains the sequence of commands and related settings that CodeBuild uses to run the build. It is broken down into phases, including:

- `install` phase sets up dependencies.
- `build` phase runs the application build.
- `artifacts` declare what to output for the next stage.

The code snippet shows an example of `buildspec.yml`:

index.yaml YAML

```yaml
version: 0.2
phases:
  install:
    commands:
      - echo Installing dependencies...
      - npm install
  build:
    commands:
      - echo Building project...
      - npm run build
artifacts:
  files:
    - '**/*'
```

An example of buildspec.yml file

Build environment

The build environment is the Docker container that CodeBuild provisions to run the build. It represents a combination of an operating system, a programm language runtime such as Node.js, Java, or Python, and the necessary tools like the AWS CLI and Docker.

Ask

Build project

The build project is the central configuration for a build. It acts as a blueprint for CodeBuild. Here are the rephrased descriptions of a CodeBuild project's configuration:

- **Source:** Defines where to get the source code, such as a repository in AWS CodeCommit or GitHub, or an archive in an S3 bucket.
- **Build Environment:** The Docker image used for the build. We can select a managed image provided by CodeBuild or specify a custom Docker image.
- **Buildspec:** The build instructions. By default, CodeBuild looks for a `buildspec.yml` file in the source code's root, but we can override this to use a different file name or define the commands directly in the project.
- **Artifacts:** The destination for the build output, typically a specific folder in an S3 bucket. We can also select "No artifacts" if the build process doesn't produce files to be stored.

AWS CodeBuild orchestrates a build by using the build project as its main configuration. Once the environment is ready, CodeBuild downloads the source code and executes the commands listed in the `buildspec.yml` file found within the code. As it runs, this process compiles code and runs tests, ultimately uploading the final software packages, known as artifacts, to an Amazon S3 bucket while streaming logs to Amazon CloudWatch.

Benefits of CodeBuild

Here are the key benefits of AWS CodeBuild:

- **Fully Managed and Serverless:** We don't need to provision, manage, or patch any build servers. CodeBuild automatically handles the underlying infrastructure and scales on demand to run our builds concurrently.
- **Pay-as-you-go Pricing:** It is highly cost-effective. We are only charged for the compute resources we use during the build, down to the second. We

Ask

don't pay for idle server time.

- **Deep AWS Integration:** It seamlessly integrates with the AWS ecosystem, including CodePipeline for orchestration, CodeCommit/GitHub for source, S3 for artifacts, and ECR for Docker images, simplifying the creation of CI/CD pipelines.

AWS CodeDeploy: Streamlining application deployments

AWS CodeDeploy automates application deployment to Amazon EC2, AWS Lambda, and Amazon ECS. It can deploy multiple types of application content, including code, Lambda functions, web and configuration files, executables, packages, scripts, multimedia files, and more. Simply put it can deploy anything that is stored on an S3 bucket, GitHub repositories, or Bitbucket repositories.

The key benefits of CodeDeploy include:

- **Automated and Centralized Deployments:** CodeDeploy fully automates the application deployment process, eliminating error-prone manual procedures. It gives us a central place to manage and monitor the status of deployments across thousands of instances, containers, or Lambda functions.
- **Minimized Application Downtime:** It supports advanced deployment strategies like in-place rolling updates and blue/green deployments. These strategies ensure the application remains available and highly available to users even while an update is in progress.
- **Increased Safety with Automatic Rollbacks:** CodeDeploy monitors the health of the application during a deployment. If the failure rate exceeds a threshold, it can automatically stop the deployment and roll back the changes to the last known stable version, preventing widespread outa from a faulty deployment.

Ask

For deployment, AWS CodeDeploy requires `appspec.yml`. It acts as the primary instruction manual, telling CodeDeploy exactly how to deploy an application to a target environment

The code snippet below shows the sample `appspec.yml` for deployment in an EC2 instance:

index.yaml YAML

```yaml
version: 0.0
os: linux
files:
  - source: /index.html
    destination: /var/www/html/
  - source: /scripts/
    destination: /opt/my-app/scripts/
hooks:
  BeforeInstall:
    - location: scripts/stop_server.sh
      timeout: 300
      runas: root
  ApplicationStart:
```

Sample appspec.yml file for EC2 instance

The `AppSpec.yml` file is structured to define two main things: what files to copy and what scripts to run during the deployment defined in the following sections of the file:

- `files`: It specifies which source files from the application revision should be copied and where they should be placed on the target instance
  - `source`: The location of the file or directory in the source bundle.
  - `destination`: The absolute path on the target instance where th should be copied

Ask

- `hooks`: It defines the sequence of scripts or commands to be executed at different life cycle events during the deployment process.

During deployment, CodeDeploy looks for the current event in the `hooks` section, fetches the related commands and executes them.

Lambda deployments use a different `appspec.yml` schema. Developers are expected to understand how life cycle events trigger function updates and how to test deployments using CloudWatch and Lambda aliases.

CI/CD workflow execution

While the design of CI/CD pipelines is out of scope, the DVA-C02 exam requires familiarity with using existing CI/CD workflows that leverage AWS services like CodeBuild and CodeDeploy. Here is a typical workflow followed by a CI/CD pipeline:

1. Developer commits code to a repository (e.g., GitHub).
2. A pipeline is triggered automatically or manually.
3. CodeBuild compiles and tests the code.
4. CodeDeploy deploys the build to a compute environment.

Following is a sample dev environment setup for better understanding:

- CodeBuild project configured with GitHub source.
- `buildspec.yml` stored at the root of the repo.
- CodeDeploy application and deployment group configured to update a Lambda function using aliases.

A typical integration flow would look something like this:

Ask

Infrastructure of a CI/CD pipeline

These tools are tightly integrated with AWS Identity and Access Management (IAM), CloudWatch for observability, and CodePipeline for orchestration.

Best Practices

The following are some best practices to follow when working with AWS CodeBuild and AWS CodeDeploy:

- Use IAM roles with least privilege for CodeBuild and CodeDeploy.
- Encrypt build artifacts and deployment packages using AWS KMS.
- Monitor logs and metrics with Amazon CloudWatch.
- Validate deployment success through Lambda versions and aliases or EC2 health checks.
- Store secrets and config values securely using Secrets Manager or Parameter Store.

Conclusion

This lesson covered two essential AWS Developer Tools: *CodeBuild* and *CodeDeploy*. We discussed how CodeBuild automates the build and testing process using a `buildspec.yml` file, and how CodeDeploy handles deployments across EC2, Lambda, or ECS. While designing pipelines is out of scope for the

Ask